САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Крамарь Кирилл

Группа K3339

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать полностью работоспособный backend API для сервиса обмена рецептами на стеке `TypeScript + Express + TypeORM + PostgreSQL + JWT + Swagger` на базе выданного бойлерплейта.

Требования к результату:

- реализовать модели, контроллеры и представления (DTO/схемы ответов);
- обеспечить авторизацию по JWT и разграничение доступа;
- реализовать ключевые ручки домена рецептов (профиль, рецепты, комментарии, лайки, избранное);
- обеспечить автодокументирование через Swagger/OpenAPI;
- получить API, которое можно поднять локально и протестировать через Postman.

Ход работы

1. Базовая инициализация и архитектурный каркас

Изначально в проекте был минимальный бойлерплейт с неполным подключением контроллеров и базовой сущностью пользователя.

Что было сделано в базовом каркасе:

- настроен единый API-префикс `APP_API_PREFIX=/api/v1`;
- подключены все реализованные контроллеры в `src/app.ts`;
- добавлен централизованный middleware обработки ошибок;
- включен Swagger UI на `/docs`;
- запуск приложения переведен на `ts-node-dev` в `npm run dev`.

Почему изменен `dev`-скрипт:

- при запуске через `tsx` библиотека `routing-controllers-openapi` падала при чтении metadata (`Reflect.getMetadata`),
- `ts-node-dev` корректно поднимает runtime metadata для декораторов в текущей связке зависимостей.

Итоговая структура слоев:

- `controllers` — HTTP-ручки и orchestration;
- `dto` — входные/выходные контракты + валидация;
- `models` — TypeORM сущности и связи;

- `mappers` — преобразование Entity -> DTO;
- `middlewares` — auth + global error handler;
- `config` — env/settings/data-source;
- `seeds` — первичное заполнение справочников.

2. Конфигурация окружения и инфраструктура

Добавлены и документированы переменные окружения:

- приложение: `APP_HOST`, `APP_PORT`, `APP_PROTOCOL`, `APP_API_PREFIX`;
- БД: `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`;
- JWT: `JWT_SECRET_KEY`, `JWT_TOKEN_TYPE`, `JWT_ACCESS_TOKEN_LIFETIME`;
- контейнер PostgreSQL: `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`.

Сделан `.env.example` как эталон конфигурации.

Docker:

- зафиксирован образ `postgres:16` в `docker-compose.yml`.

Причина фикса версии:

- при `postgres:latest` (18+) контейнер мог уходить в restart-loop при существующем каталоге данных (`dbs/postgres-data`),
- фиксация на 16 устранила несовместимость хранения и стабилизировала локальный запуск.

3. Реализация модели данных (TypeORM)

Реализованы сущности:

- `User`
- `RecipeCategory`
- `DifficultyLevel`
- `Ingredient`
- `Recipe`
- `RecipeIngredient`
- `RecipeStep`
- `RecipeMedia`
- `Comment`
- `RecipeLike`
- `Favorite`

Ключевые связи и ограничения:

- `User 1:N Recipe`
- `RecipeCategory 1:N Recipe`
- `DifficultyLevel 1:N Recipe`
- `Recipe M:N Ingredient` через `RecipeIngredient`
- `Recipe 1:N RecipeStep`
- `Recipe 1:N RecipeMedia`
- `Recipe 1:N Comment`, `User 1:N Comment`
- `User M:N Recipe` через `RecipeLike`
- `User M:N Recipe` через `Favorite`

Уникальные ограничения:

- `users.username`, `users.email`
- `recipe_likes(recipe_id, user_id)`
- `favorites(recipe_id, user_id)`
- `recipe_ingredients(recipe_id, ingredient_id)`
- `recipe_steps(recipe_id, step_number)`
- `recipe_media(recipe_id, sort_order)`

Каскады удаления:

- удаление рецепта удаляет дочерние `ingredients/steps/media/comments/likes/favorites`.

Технические нюансы:

- в `DataSource` используется `synchronize: true` (удобно для лабораторной, но не для production);
- `logging: true` оставлен включенным для прозрачности SQL на защите.

4. DTO, валидация и контракты ответов

Реализован единый набор DTO:

- Auth: регистрация/логин/ответ авторизации;
- Users: профиль, update profile, change password;
- Recipes: create/update/query/list/detail;
- Comments: create/list response;
- Reactions: response;
- Pagination: универсальные paginated-response DTO.

Валидация сделана через `class-validator`:

- длины строк, email/url, enum, min/max, целые/десятичные значения;

- длины строк, email/url, enum, min/max, целые/десятичные значения;
- валидация массивов ингредиентов/шагов/медиа;
- проверка дублирующихся `ingredientId/stepNumber/sortOrder` на уровне бизнес-логики.

Мапперы:

- введен слой `Entity -> DTO`, чтобы не отдавать чувствительные поля (`passwordHash`) и держать единый wire-format.

5. Аутентификация и безопасность

Auth-логика:

- `POST /auth/register`
- `POST /auth/login`

Решения:

- хранение пароля только в хеше (`bcrypt`),
- JWT с payload `{ user: { id } }`,
- `Authorization: Bearer <token>` для защищенных ручек,
- унифицированный `tokenType` и `expiresIn` в ответе.

Права доступа:

- редактирование/удаление рецепта только владельцем (`403` при чужом ресурсе),
- защищенные ручки требуют валидный токен (`401` при отсутствии/битом токене).

6. Реализация API-ручек

6.1 Auth

- `POST /api/v1/auth/register`
- `POST /api/v1/auth/login`

6.2 Users

- `GET /api/v1/users/me`
- `PATCH /api/v1/users/me`
- `PATCH /api/v1/users/me/password`

6.3 Recipes

- `POST /api/v1/recipes`
- `GET /api/v1/recipes`

- `GET /api/v1/recipes/:recipeId`
- `PATCH /api/v1/recipes/:recipeId`
- `DELETE /api/v1/recipes/:recipeId`

Поддержка query для списка:

- `q`, `categoryId`, `difficultyId`, `ingredientId`, `authorId`, `sortBy`, `sortOrder`, `page`, `size`.

6.4 Comments

- `GET /api/v1/recipes/:recipeId/comments`
- `POST /api/v1/recipes/:recipeId/comments`

6.5 Likes/Favorites

- `POST /api/v1/recipes/:recipeId/like`
- `DELETE /api/v1/recipes/:recipeId/like`
- `POST /api/v1/recipes/:recipeId/favorite`
- `DELETE /api/v1/recipes/:recipeId/favorite`

6.6 Reference data

- `GET /api/v1/reference-data/categories`
- `GET /api/v1/reference-data/difficulty-levels`
- `GET /api/v1/reference-data/ingredients`

7. Единая обработка ошибок

Добавлен глобальный after-middleware ошибок:

- единый формат:

```
{
  "code": "VALIDATION_ERROR",
  "message": "...",
  "details": ["..."]
}
```

- маппинг статусов в коды ошибок (`400/401/403/404/409/500`),
- разбор ошибок валидации,
- отдельная обработка `PostgreSQL unique violation (23505)` как `409 Conflict`.

8. Swagger/OpenAPI

- подключен `routing-controllers-openapi` + `swagger-ui-express`;

- схемы DTO подтягиваются через `class-validator-jsonschema`;
- security scheme `bearerAuth` добавлена в OpenAPI;
- на ручках проставлены `@OpenAPI` и `@ResponseSchema`.

Нюанс:

- у `routing-controllers-openapi` есть чувствительность к metadata параметров;
- чтобы избежать падений генератора, в `@Body` и `@QueryParams` задан явный `type`.

9. Сидинг справочников

После `dataSource.initialize()` выполняется seed (`seedReferenceData`):

- категории,
- уровни сложности,
- ингредиенты.

Сид происходит только если соответствующие таблицы пусты.

10. Отладка и устраненные проблемы

Проблема 1: Swagger падал на старте

Симптом:

- ошибка в `routing-controllers-openapi` при чтении параметров.

Причина:

- отсутствие ожидаемой runtime metadata в текущем режиме запуска + неявные типы декораторов.

Решение:

- переход на `ts-node-dev` для dev-режима,
- явный `type` в `@Body({...})` и `@QueryParams({...})`.

Проблема 2: `ECONNREFUSED` к БД

Симптом:

- API не мог подключиться к `localhost:15432`.

Причина:

- контейнер БД не поднимался из-за несовместимости каталога данных с `postgres:latest`.

Решение:

- фиксация образа `postgres:16`,
- пересоздание контейнера.

Проблема 3: 500 на `GET /recipes` и `GET /recipes/:id/comments`

Симптом:

- `Cannot read properties of undefined (reading 'databaseName')`.

Причина:

- `orderBy` был задан через snake_case DB-имена (`created_at`) в запросах с `distinct + pagination`.

Решение:

- использовать property-path TypeORM (`createdAt`, `cookingTimeMinutes`) вместо физических имен колонок.

Проблема 4: `EADDRINUSE: 8000`

Симптом:

- повторный запуск падал с занятым портом.

Причина:

- старый dev-процесс не был остановлен.

Решение:

- завершение старого процесса (`pkill` / `lsof + kill`).

11. Как проверялось

Основной сценарий проверки через Postman:

1. Регистрация -> получение токена.
2. Логин -> проверка токена.
3. `GET /users/me`.
4. Чтение справочников.
5. Создание рецепта с ингредиентами/шагами.
6. Список рецептов (пагинация/фильтры/сортировка).
7. Получение одного рецепта.
8. Комментарий к рецепту + список комментариев.

9. Лайк/снятие лайка.
10. Избранное/удаление из избранного.
11. Обновление и удаление рецепта владельцем.
12. Проверка конфликтов (`409`) и прав доступа (`403`).

Вывод

В рамках работы был реализован полноценный MVP backend API сервиса рецептов на заявленном стеке.

Достигнуты основные цели:

- реализована предметная модель (пользователи, рецепты, ингредиенты, шаги, медиа, комментарии, лайки, избранное);
- построены защищенные JWT-ручки с валидацией и проверкой прав;
- добавлены пагинация/фильтрация/сортировка;
- обеспечена автодокументация через Swagger;
- подготовлена инфраструктура локального запуска (`docker compose` , `.env.example` , подробный README);
- устранены критические runtime-проблемы (Swagger metadata, несовместимость Postgres latest, 500 в пагинации TypeORM).

Итог: API стабильно поднимается локально, воспроизводимо тестируется в Postman и подходит для демонстрации на защите с техническими пояснениями по архитектуре, безопасности и отладке.